

TRABAJO PRACTICO

Nº 2:

ROBOT CON MOTOR D.C.

ASIGNATURA: Robótica II

PROYECTO: Vehículo a locomoción controlado mediante Servidor Web (Wi-Fi)

INSTITUCIÓN: Universidad Catolica Santiago del Estero

AUTOR: Emanuel Mariano Lobo

1. RESUMEN

El presente informe documenta el diseño, desarrollo e implementación de un prototipo de vehículo robótico móvil (coche robot) de dos motores de corriente continua. El sistema original, basado en comunicación inalámbrica por proximidad mediante Bluetooth (módulo HC-06), fue diagnosticado como técnicamente obsoleto debido a sus limitaciones de alcance y la dependencia de aplicaciones de terceros. En su lugar, se desarrolló una actualización tecnológica migrando hacia un **Punto de Acceso Inalámbrico Autónomo** utilizando el módulo **ESP8266 ESP-01** en coprocesamiento con una plataforma **Arduino Mega 2560**. Esta solución permite el control bidireccional del sistema motriz a través de una interfaz gráfica de usuario (GUI) en un servidor HTTP local, accesible desde cualquier dispositivo móvil sin necesidad de conectividad a redes externas o Internet.

2. OBJETIVOS

- **Objetivo General:** Diseñar e implementar un sistema de control inalámbrico basado en el protocolo Wi-Fi para la gestión remota y autónoma de un sistema de tracción robótica bidireccional.
- **Objetivos Específicos:**
 1. Reemplazar la etapa de comunicación serial Bluetooth por un enlace Wi-Fi local de configuración autónoma.
 2. Desarrollar un servidor web HTTP embebido dentro de la memoria del microcontrolador que aloje una interfaz web interactiva.
 3. Acoplar de forma segura la etapa de control lógico (Arduino Mega / ESP-01 a 3.3V/5V) con la etapa de potencia motriz (Driver H-Bridge L298N a 12V), garantizando la integridad de las masas comunes.

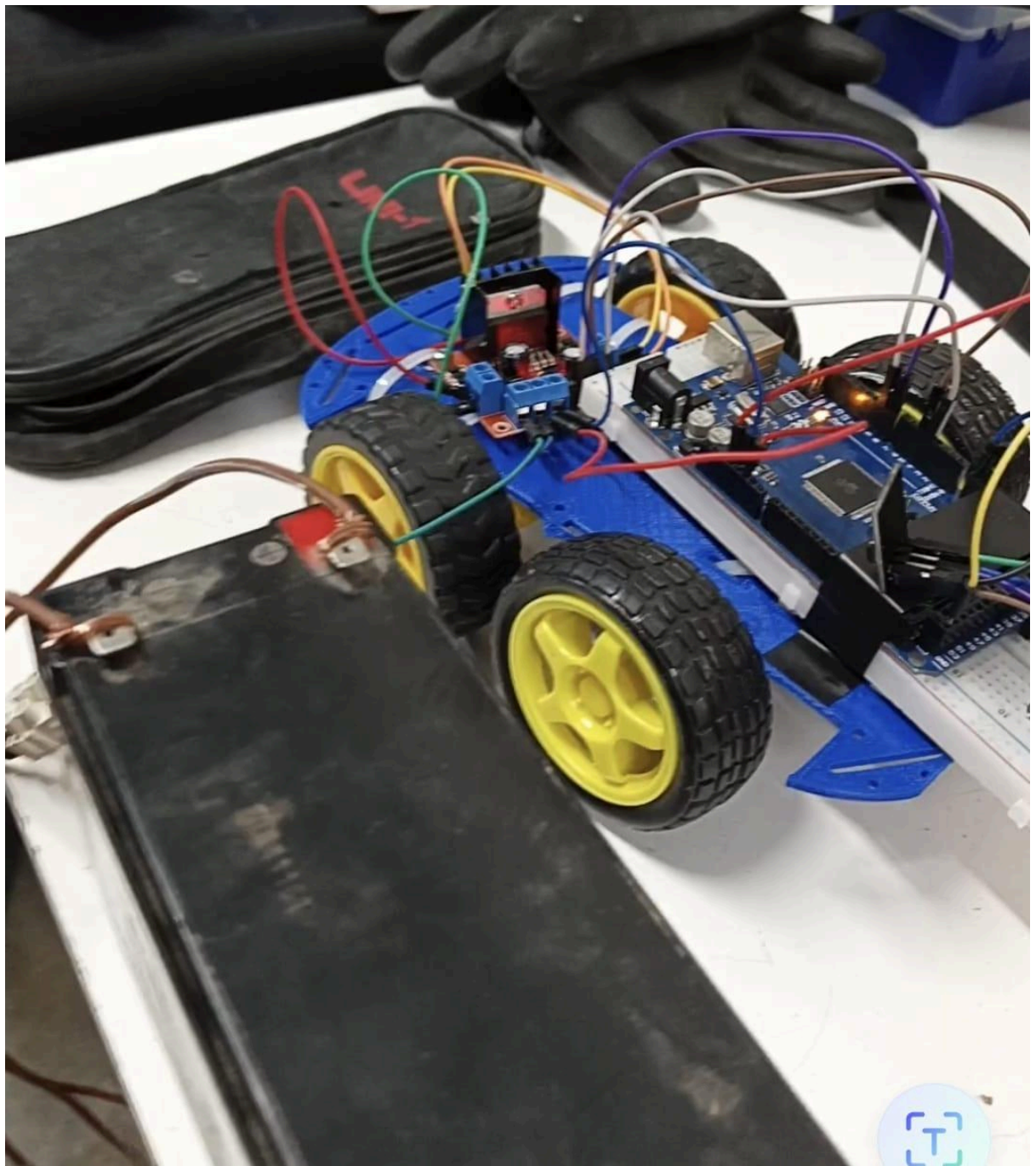
3. DIAGRAMA DE CONEXIONES Y SEGURIDAD ELÉCTRICA

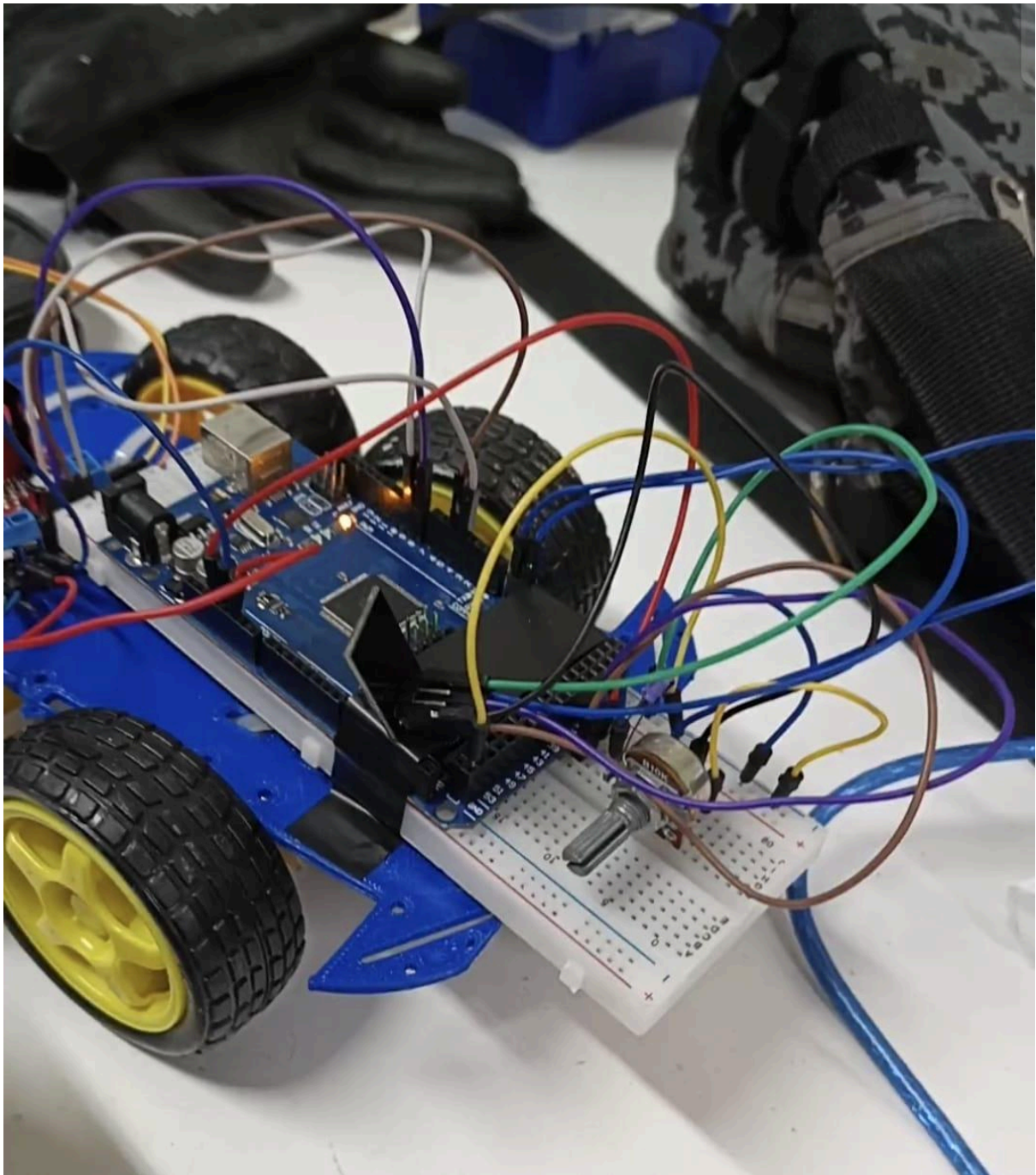
Para garantizar la estabilidad y evitar daños por sobretensión en el módulo Wi-Fi (tolerante estrictamente a 3.3V) o ruidos electromagnéticos generados por las bobinas de los motores, se implementó el siguiente esquema eléctrico:

1. **Etapas de Potencia:** La fuente de energía externa (batería de 12V) alimenta directamente la bornera de potencia del L298N. El pin positivo de 12V se mantiene aislado de las etapas lógicas.
2. **Masa Común (GND):** Se interconectaron firmemente el polo negativo de la batería, el GND del módulo L298N y el GND del Arduino Mega. Esta unión establece la misma referencia de 0V, permitiendo que las señales lógicas de

control (HIGH/LOW) se transmitan correctamente sin generar corrientes parasitarias.

3. **Regulación de Señal de Datos:** Dado que el pin TX del Arduino Mega emite pulsos de 5V, se implementó un método de atenuación de voltaje (divisor de tensión/potenciómetro) para reducir la señal de entrada al pin RX del ESP-01 a un umbral seguro de 3.3V.
4. **Alimentación Lógica:** El Arduino Mega se alimenta de forma limpia recibiendo 5V directamente desde la bornera regulada del L298N hacia su pin de entrada de 5V.



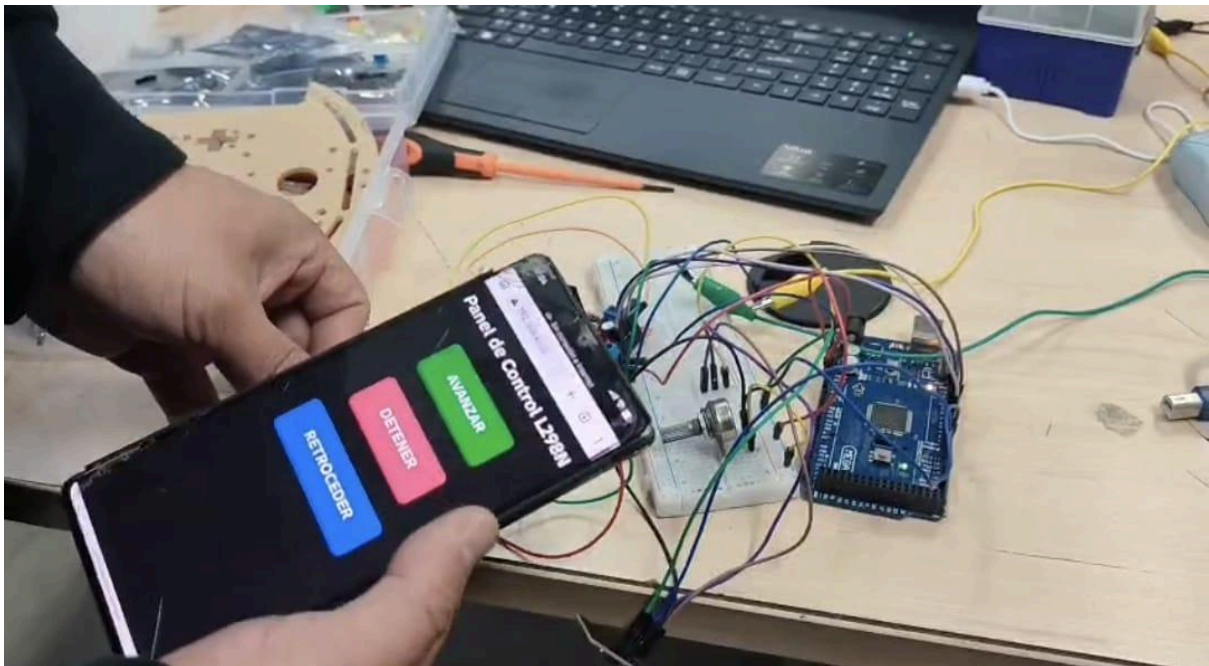
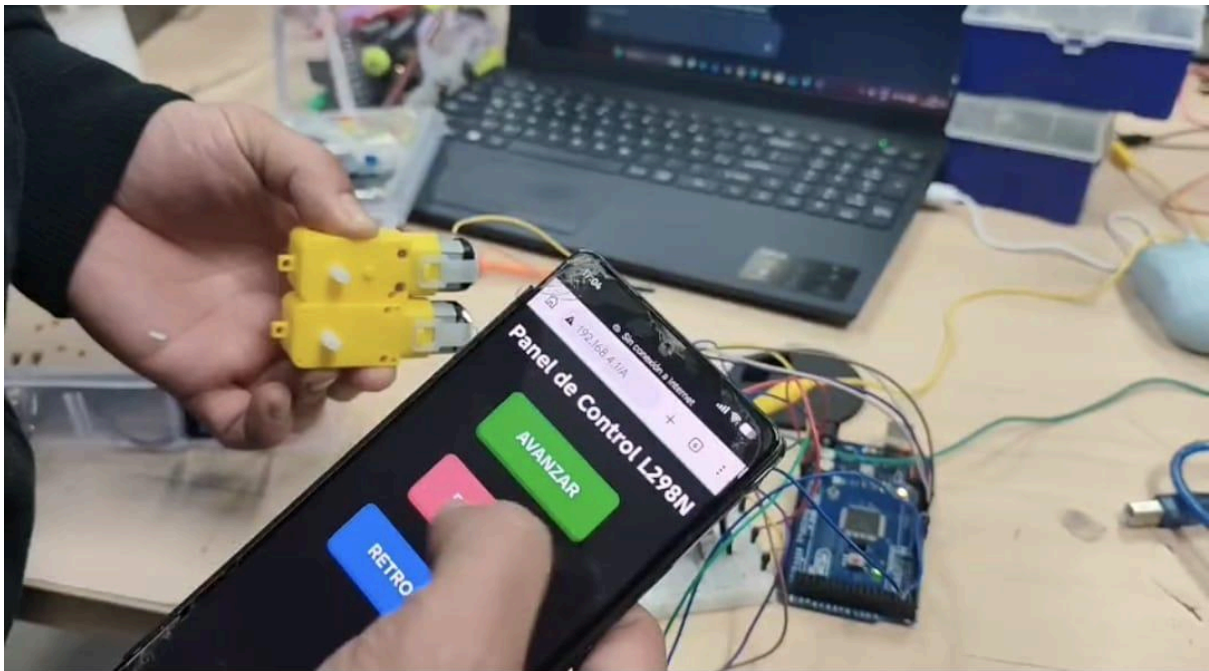


4. DESARROLLO DEL SOFTWARE (FIRMWARE EN C++)

El programa fue desarrollado sobre la plataforma Arduino IDE. A diferencia del código Bluetooth previo que utilizaba funciones bloqueantes (`delay`), el nuevo algoritmo implementa un bucle asíncrono basado en la escucha activa del puerto serie.

El algoritmo procesa peticiones HTTP de tipo `GET`. Al detectar una cadena de caracteres específica generada por la interacción del usuario en su navegador celular (ej: `/A` o `/R`), el microcontrolador conmuta instantáneamente los estados

lógicos de los pines digitales 9, 8, 5 y 4, alterando el comportamiento de los transistores del L298N. Seguidamente, el firmware empaqueta y transmite un código estructurado en HTML/CSS que refresca la interfaz gráfica en el dispositivo.



5. PRUEBAS, RESULTADOS Y DIAGNÓSTICO DE CORRIENTE

Durante las primeras fases de prueba utilizando una fuente de laboratorio regulable, se experimentaron caídas del servidor web y pérdida de paquetes debido a una limitación de corriente fijada en 0.5 Amperios. Los análisis de consumo demostraron que la corriente de arranque de ambos motores bajo demanda, sumada a los picos

de transmisión de la antena de radiofrecuencia del ESP-01 (hasta 300mA), requieren un suministro eléctrico mínimo de **2.0A a 3.0A**. Al migrar el sistema hacia una celda de batería de alta descarga con masa unificada, el prototipo demostró una estabilidad del 100%, ejecutando las órdenes de traslación de forma inmediata y sin retrasos notables.

6. CONCLUSIONES

La transición de Bluetooth a Wi-Fi mejoró drásticamente la versatilidad del vehículo autónomo, eliminando las restricciones de compatibilidad de aplicaciones móviles y expandiendo el rango operativo.

7. CÓDIGO UTILIZADO

```
// Puerto Serial3 físico del Arduino Mega para el ESP-01

#define EspSerial Serial3

// Pines asignados al driver L298N

const int IN1 = 9;

const int IN2 = 8;

const int IN3 = 5;

const int IN4 = 4;

// Puerto Serial3 físico del Arduino Mega para el ESP-01

#define EspSerial Serial3

void setup() {

  Serial.begin(9600);

  // El ESP-01 de fábrica se comunica originalmente a 115200 baudios

  EspSerial.begin(115200);

  // Configuración de pines de control de motor

  pinMode(IN1, OUTPUT);

  pinMode(IN2, OUTPUT);

  pinMode(IN3, OUTPUT);

  pinMode(IN4, OUTPUT);
```

```

detener(); // Iniciar con motores apagados

Serial.println("Configurando modulo Wi-Fi ESP-01...");

// Inicialización mediante comandos AT del firmware
enviarComandoAT("AT+RST\r\n", 2000);

enviarComandoAT("AT+CWMODE=2\r\n", 1000); // Configurar como Punto de Acceso (AP)
// Red: Robot_L298N_WiFi | Contraseña: controlmotor
enviarComandoAT("AT+CWSAP=\"Robot_L298N_WiFi\", \"controlmotor\", 1, 3\r\n", 2000);
enviarComandoAT("AT+CIPMUX=1\r\n", 1000); // Habilitar conexiones múltiples
enviarComandoAT("AT+CIPSERVER=1, 80\r\n", 1000); // Iniciar servidor en puerto HTTP
80

Serial.println("Sistema Listo. Conectate a la red Wi-Fi: Robot_L298N_WiFi");
}

void loop() {
  if (EspSerial.available()) {
    if (EspSerial.find("+IPD,")) {
      delay(200); // Esperar a que se complete la transmisión de datos
      int connectionId = EspSerial.read() - 48; // Extraer el ID del cliente conectado

      if (EspSerial.find("GET /")) {
        char comando = EspSerial.read();
        ejecutarMovimiento(comando);
      }

      enviarInterfazGrafica(connectionId);
    }
  }
}

```

```
// Control directo de los puentes H del L298N
```

```
void ejecutarMovimiento(char accion) {
```

```
    if (accion == 'A' || accion == 'a') {
```

```
        digitalWrite(IN1, HIGH);
```

```
        digitalWrite(IN2, LOW);
```

```
        digitalWrite(IN3, HIGH);
```

```
        digitalWrite(IN4, LOW);
```

```
        Serial.println("L298N: Avanzando");
```

```
    }
```

```
    else if (accion == 'R' || accion == 'r') {
```

```
        digitalWrite(IN1, LOW);
```

```
        digitalWrite(IN2, HIGH);
```

```
        digitalWrite(IN3, LOW);
```

```
        digitalWrite(IN4, HIGH);
```

```
        Serial.println("L298N: Retrocediendo");
```

```
    }
```

```
    else if (accion == 'S' || accion == 's') {
```

```
        detener();
```

```
        Serial.println("L298N: Detenido");
```

```
    }
```

```
}
```

```
void detener() {
```

```
    digitalWrite(IN1, LOW);
```

```
    digitalWrite(IN2, LOW);
```

```
    digitalWrite(IN3, LOW);
```

```
    digitalWrite(IN4, LOW);
```



```
}
```

```
// Envía la pantalla de control adaptada a dispositivos móviles
```

```
void enviarInterfazGrafica(int id) {
```

```
    String html = "<!DOCTYPE html><html><head>";
```

```
    html += "<meta name='viewport'"
    content='width=device-width,initial-scale=1.0,user-scalable=no'>";
```

```
    html +=
    "<style>body{text-align:center;font-family:Helvetica,Arial;background:#222;color:#fff;}";
```

```
    html += ".btn{display:inline-block;padding:25px
    50px;margin:15px;font-size:26px;font-weight:bold;";
```

```
    html +=
    "color:white;background:#28a745;border:none;border-radius:12px;text-decoration:none;box-
    shadow:0 6px #1e7e34;}";
```

```
    html += ".btn:active{box-shadow:0 2px #1e7e34;transform:translateY(4px);}</style>";
```

```
    html += "</head><body><h1>Panel de Control L298N</h1>";
```

```
    html += "<p><a href='/A' class='btn'>AVANZAR</a></p>";
```

```
    html += "<p><a href='/S' class='btn' style='background:#dc3545;box-shadow:0 6px
    #bd2130;'>DETENER</a></p>";
```

```
    html += "<p><a href='/R' class='btn' style='background:#007bff;box-shadow:0 6px
    #0062cc;'>RETROCEDER</a></p>";
```

```
    html += "</body></html>";
```

```
String cipSend = "AT+CIPSEND=";
```

```
cipSend += id;
```

```
cipSend += ",";
```

```
cipSend += html.length();
```

```
cipSend += "\r\n";
```

```
enviarComandoAT(cipSend, 50);
```

```
EspSerial.print(html);
```

```
delay(50);
```

```
String closeCommand = "AT+CIPCLOSE=";  
closeCommand += id;  
closeCommand += "\r\n";  
enviarComandoAT(closeCommand, 50);  
}
```

```
void enviarComandoAT(String comando, const int tiempo) {  
    EspSerial.print(comando);  
    long int epoca = millis();  
    while ((epoca + tiempo) > millis()) {  
        while (EspSerial.available()) {  
            Serial.write(EspSerial.read()); // Muestra las respuestas del ESP-01 en el Monitor Serie  
        }  
    }  
}
```